

DOCUMENTO TECNICO FINAL

Proyecto: Prueba Tecnica Especialista de Datos (Proceso ETL y Visualizacion)

Ultima actualizacion: 2026-07-28

Estado: version final para entrega

1. INTRODUCCION

Este documento describe la solucion implementada para la prueba tecnica de Especialista de Datos. El alcance cubre la carga de archivos .xlsb, la transformacion y depuracion de datos, el diseno de un Data Warehouse local en PostgreSQL, la orquestacion con Apache Airflow y la construccion de un tablero ejecutivo en Power BI.

La solucion se diseno para responder dos retos de negocio:

- Seguimiento al desempeno comercial de ventas frente a presupuesto.
- Seguimiento a la gestion de registros TMK Outbound.

2. OBJETIVOS DEL PROYECTO

El objetivo tecnico fue implementar una base de datos local que funcionara como fuente confiable para Power BI, evitando analisis directos sobre archivos Excel y separando claramente las responsabilidades entre extraccion, transformacion, persistencia y visualizacion.

El objetivo analitico fue permitir al Director Comercial monitorear:

- Ventas validas frente al presupuesto.
- Desempeno por region, aliado, gerente, jefe y especialista.
- Gestion TMK por aliado, campana y segmento.
- Contactabilidad, contactos efectivos y contactos no efectivos.

3. ARQUITECTURA DE LA SOLUCION

La arquitectura implementada sigue este flujo:

Archivos .xlsb

- > Python ETL
- > PostgreSQL local en Docker
- > Apache Airflow para orquestacion
- > Power BI Desktop

Componentes principales:

- data/raw/: fuentes originales Ventas.xlsb, Presupuesto.xlsb y Registros.xlsb.
- etl/: extraccion, transformacion, reglas de negocio, carga y validacion.
- sql/ddl/ y sql/dml/: creacion del esquema fisico y seeds.
- repository/: persistencia desacoplada hacia PostgreSQL.
- airflow/dags/: DAGs de ejecucion.
- powerbi/PowerBI_ETL_Comercial.pbix: tablero ejecutivo.

Airflow solo orquesta. La logica de negocio reside en Python y el consumo analitico se realiza desde Power BI contra PostgreSQL.

4. DISEÑO DEL MODELO DE DATOS

Se implemento un modelo dimensional orientado a analisis, con tres procesos de negocio:

- Ventas comerciales.
- Presupuesto / metas comerciales.
- Gestion TMK Outbound.

Hechos:

- dwh.fact_ventas
- dwh.fact_presupuesto
- dwh.fact_gestion_tmk

Dimensiones:

- dwh.dim_tiempo
- dwh.dim_region
- dwh.dim_canal
- dwh.dim_categoria
- dwh.dim_jerarquia_comercial
- dwh.dim_aliado
- dwh.dim_unidad_gestion
- dwh.dim_marca
- dwh.dim_vendedor
- dwh.dim_validez_venta
- dwh.dim_campana
- dwh.dim_segmento
- dwh.dim_tipo_contacto

Granos principales:

- Ventas: una fila representa un evento transaccional de venta.
- Presupuesto: una fila representa una meta mensual en un cruce comercial.
- Gestion TMK: una fila representa un agregado de gestion por tipificacion.

5. JUSTIFICACION DEL MODELO IMPLEMENTADO

El enunciado solicita evaluar si para este escenario resulta mas adecuada una arquitectura tipo estrella (Star Schema) o copo de nieve (Snowflake Schema). Para la solucion implementada se eligio Star Schema como arquitectura principal.

La razon principal es que el objetivo del proyecto es analitico y ejecutivo: Power BI debe responder rapidamente preguntas de negocio sobre ventas, presupuesto y gestion TMK. El modelo estrella reduce la cantidad de saltos entre tablas, simplifica las relaciones y facilita que los usuarios consuman dimensiones de negocio directamente desde el panel de campos.

En este caso existen tres hechos (fact_ventas, fact_presupuesto, fact_gestion_tmk) que comparten dimensiones conformadas como tiempo, region, aliado y jerarquia comercial. Esta estructura encaja naturalmente con un Star Schema porque cada hecho se conecta de forma directa con sus dimensiones, sin depender de cadenas largas de relaciones.

Se evaluo Snowflake Schema como alternativa, especialmente para jerarquias como canal, region o

jerarquía comercial. Sin embargo, se descarto porque normalizar esas dimensiones en subdimensiones aumentaría la complejidad del modelo semántico, haría más difícil la navegación en Power BI y agregaría relaciones adicionales sin un beneficio proporcional para el volumen y el tipo de análisis requerido.

Desde buenas prácticas de rendimiento en Power BI, el Star Schema ofrece ventajas claras:

- Menos relaciones activas que evaluar durante los filtros.
- Relaciones 1:N directas desde dimensiones hacia hechos.
- Tablas de hechos enfocadas en medidas y llaves surrogate.
- Dimensiones con atributos descriptivos listos para filtros, segmentadores y jerarquías.
- Modelo semántico más simple para el usuario final.

El modelo implementado en Power BI refleja la estructura dimensional creada en PostgreSQL. Las tablas `dwh.fact_*` se mantienen como hechos y las tablas `dwh.dim_*` como dimensiones. Las relaciones se configuraron desde dimensiones hacia hechos usando las claves `sk_*`, conservando la integridad del Data Warehouse.

Para mejorar la experiencia de usuario, las claves técnicas `sk_*` y columnas de auditoría como `fecha_carga_dw` se ocultaron en la vista de informe. De esta forma, el usuario de negocio ve atributos analíticos como `region`, `aliado`, `gerente`, `jefe`, `especialista`, `campana`, `segmento` y medidas DAX, sin exponerse a llaves técnicas del modelo.

En conclusión, Star Schema es la opción más adecuada para este escenario porque equilibra rendimiento, claridad, trazabilidad con la base de datos y facilidad de uso en Power BI. Snowflake Schema se consideró, pero se descarto para evitar complejidad innecesaria en un tablero ejecutivo orientado a análisis comercial.

6. MODELO FISICO

El modelo físico se implementó en PostgreSQL bajo el esquema `dwh`.

Características:

- Surrogate keys `sk_*` en dimensiones y hechos.
- Relaciones 1:N desde dimensiones hacia hechos.
- Índices analíticos sobre claves de hechos.
- Miembro `sk=0` para valores no informados.
- Tabla `dim_validez_venta` para materializar la regla de ventas válidas.
- `dim_tiempo` como calendario corporativo con festivos Colombia y bandera `es_habil`.

La carga de hechos usa estrategia Full Load: `TRUNCATE` + carga completa. Para soportar el volumen real, la carga de hechos se optimizó con PostgreSQL `COPY FROM STDIN`.

7. DEPURACIONES Y TRANSFORMACIONES

Transformaciones aplicadas:

- Normalización de nombres de columnas.
- Limpieza de textos y espacios.
- Conversión de fechas y periodos.
- Conversión de medidas numéricas.
- Deduplicación técnica.
- Homologación de catálogos dimensionales.
- Asignación de miembros No informado.
- Construcción de dimensiones conformadas.

- Resolucion de surrogate keys para hechos.

Reglas de negocio destacadas:

- Venta valida: tiene codigo de factura y no presenta nota credito.
- TMK Outbound: se conserva como universo de analisis de Registros.xlsb.
- Tipificacion TMK: contactos efectivos, contactos no efectivos y no contactos.
- Presupuesto: se conserva el total T&T como medida monetaria de presupuesto.
- Calendario: dias habiles lunes a sabado, excluyendo festivos nacionales de Colombia segun dim_tiempo.

8. IMPLEMENTACION DEL ETL

El ETL esta organizado por capas:

- etl/extract/: lectura de archivos .xlsb.
- etl/transform/: limpieza tecnica por dataset.
- etl/business_rules/: reglas de negocio.
- etl/load/: orquestacion de carga de dimensiones y hechos.
- repository/: acceso a PostgreSQL.
- etl/validation.py: validacion del Data Warehouse.

La ejecucion final se realizo sin limite de muestra (max_rows=None) y cargo:

- fact_ventas: 243359 filas.
- fact_presupuesto: 731 filas.
- fact_gestion_tmk: 459567 filas.

Resultado de validacion:

- Validacion DW: OK.
- Chequeos: 38.
- Fallidos: 0.
- Orphans: 0.

9. ORQUESTACION CON APACHE AIRFLOW

Se implemento un DAG principal para ejecutar el pipeline comercial:

- airflow/dags/etl_comercial_pipeline.py

Caracteristicas:

- DAG manual (schedule=None).
- Una tarea principal run_etl_pipeline.
- PythonOperator.
- Reintentos configurados.
- Timeout operativo.
- Logging en consola y archivo.

Tambien se implemento infraestructura para calendario corporativo mediante calendar_seed_dag, que alimenta dim_tiempo con fechas, festivos y dias habiles.

10. BASE DE DATOS POSTGRESQL

La base local se ejecuta con Docker Compose y PostgreSQL 16. La configuracion se maneja por variables de entorno a partir de replace.env y .env.

Validaciones implementadas:

- Existencia del esquema dwh.
- Existencia de 13 dimensiones.
- Existencia de 3 hechos.
- Conteos por tabla.
- Integridad referencial.
- Miembros sk=0.
- Dominio de dim_validez_venta.
- Confirmacion de que fact_gestion_tmk no contiene sk_canal.

11. DASHBOARD POWER BI

El archivo powerbi/PowerBI_ETL_Comercial.pbix contiene tres paginas:

Resumen_Ejecutivo

Incluye KPIs principales:

- Ventas validas.
- Presupuesto T&T.
- Cumplimiento.
- Faltante.
- Ventas vs presupuesto por region.

Tras refrescar con datos completos:

- Ventas validas aproximadas: \$ 327 mil M.
- Presupuesto T&T aproximado: \$ 364 mil M.
- Cumplimiento aproximado: 90,06 %.
- Faltante aproximado: \$ 36 mil M.

Detalle_Comercial

Incluye matrices por:

- Aliado.
- Gerente.
- Jefe.
- Especialista.

Permite identificar sobrecumplimientos e incumplimientos mediante ventas, presupuesto, cumplimiento y faltante.

TMK_Outbound

Incluye:

- Registros entregados.
- % Gestion.
- % Contactabilidad.
- % Contactos efectivos.
- % Contactos no efectivos.
- Top 5 aliados por contactabilidad.
- Campanas con mejor contactabilidad.
- Campanas con mayor y menor cantidad de registros.
- Contactabilidad por segmento.

Tras refrescar con datos completos:

- Registros entregados aproximados: 55 mil.
- Gestion: 32,34 %.
- Contactabilidad: 18,03 %.
- Contactos efectivos: 14,16 %.
- Contactos no efectivos: 3,87 %.

12. PRINCIPALES MEDIDAS DAX

Medidas de ventas y presupuesto:

Ventas_\$ = SUM('dwh fact_ventas'[valor_antes_iva])

Ventas_Validas_\$ =
CALCULATE(
[Ventas_\$],
'dwh dim_validez_venta'[es_venta_valida] = TRUE()
)

Presupuesto_TYT = SUM('dwh fact_presupuesto'[tyt])

Cumplimiento_TYT_Valido_% =
DIVIDE([Ventas_Validas_\$], [Presupuesto_TYT])

Faltante_TYT_Valido_\$ =
[Presupuesto_TYT] - [Ventas_Validas_\$]

Medidas TMK:

Registros_Entregados = [Gestiones_TMK]

Contactos_Efectivos =
CALCULATE(
[Gestiones_TMK],
'dwh dim_tipo_contacto'[tipo_contacto] = "CONTACTOS EFECTIVOS"
)

Contactos_No_Efectivos =
CALCULATE(
[Gestiones_TMK],
'dwh dim_tipo_contacto'[tipo_contacto] = "CONTACTOS NO EFECTIVOS"
)

Contactos_TMK =
[Contactos_Efectivos] + [Contactos_No_Efectivos]

Contactabilidad_TMK_% =

$\text{DIVIDE}([\text{Contactos_TMK}], [\text{Registros_Entregados}])$

$\text{Contactos_Efectivos_}\% =$
 $\text{DIVIDE}([\text{Contactos_Efectivos}], [\text{Registros_Entregados}])$

$\text{Contactos_No_Efectivos_}\% =$
 $\text{DIVIDE}([\text{Contactos_No_Efectivos}], [\text{Registros_Entregados}])$

Las formulas exactas de % Gestion, % Contactabilidad, % Contactos efectivos y % Contactos no efectivos no estaban especificadas en el PDF; se adopto una definicion consistente con la tipificacion observada en la fuente de registros.

13. NOVEDADES IDENTIFICADAS DURANTE EL ANALISIS

Hallazgos relevantes:

- Codigo Factura no es clave natural unica de ventas.
- Ventas contiene registros sin factura y con nota credito; por eso se creo dim_validez_venta.
- Presupuesto contiene medidas monetarias terminales, tecnologia y tyt.
- Registros TMK llega en grano agregado, no como contacto unitario.
- La jerarquia comercial no tiene cobertura completa en todos los registros.
- Existen valores sin region o sin informacion asignada.
- Algunas formulas de TMK no estaban cerradas explicitamente en el enunciado.
- Se identifico PII potencial en identificaciones de vendedores, por lo que se ocultaron campos tecnicos/no necesarios en Power BI.

14. USO DE HERRAMIENTAS DE IA

Se utilizaron herramientas de IA como apoyo metodologico y de productividad:

- ChatGPT: rol de Tech Lead para planificacion, revision de decisiones, validacion funcional y guia de construccion del tablero.
- Cursor: apoyo en implementacion, revision de codigo, documentacion y actualizacion de bitacora.

Ejemplos de prompts utilizados:

- "Actua como inspector visual de Power BI y describe unicamente lo que ves".
- "Guiame como Tech Lead para finalizar el proyecto y cumplir el PDF".
- "Revisa que falta contra los requisitos de la prueba tecnica".
- "Completa el documento tecnico final con modelo, ETL, Power BI, DAX y uso de IA".

La IA no sustituyo las fuentes oficiales del proyecto. Las decisiones se contrastaron con el PDF, el perfilado de datos, los logs del ETL y el modelo implementado.

15. CONCLUSIONES

La solucion implementa un flujo completo desde fuentes .xlsb hasta un tablero ejecutivo en Power BI sobre PostgreSQL. El modelo dimensional permite analizar ventas, presupuesto y gestion TMK con dimensiones de negocio reutilizables.

El ETL final fue ejecutado con datos completos y validacion satisfactoria del Data Warehouse. Power BI fue refrescado contra la carga completa y presenta indicadores coherentes para los dos retos solicitados.

Quedan como mejoras futuras:

- Formalizar con negocio las formulas exactas de TMK no especificadas en el PDF.
- Profundizar en meta diaria dinamica y acumulacion de deficit con mas visuales especificos.
- Publicar el tablero en Power BI Service si el entorno lo permite.